

```
## 1. DB Connection — lib/db.js
import mongoose from "mongoose";
let cached = global.mongoose;
if (!cached) cached = global.mongoose = { conn: null, promise: null };

async function connectDB() {
  if (cached.conn) return cached.conn;
  if (!cached.promise)
    cached.promise =
      mongoose.connect(process.env.MONGO_URI).then((m) => m);
  cached.conn = await cached.promise;
  return cached.conn;
}

export default connectDB;
```

```
## 2. User Schema — models/User.js
import mongoose from "mongoose";
const UserSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true }, // remove if no auth
  age: { type: Number },
  { timestamps: true }
}); // adds createdAt, updatedAt

export default mongoose.models.User || mongoose.model("User",
UserSchema);
```

```
## 3. API CRUD - api/users/route.js
import { NextResponse } from "next/server";
import connectDB from "@lib/db";
import User from "@models/User";

// GET all users
export async function GET() {
  await connectDB();
  const users = await User.find();
  return NextResponse.json({ success: true, data: users }, { status:
200 });
}
```

```
// POST create user
export async function POST(request) {
  await connectDB();
  const body = await request.json();
  const user = await User.create(body);
  return NextResponse.json({ success: true, data: user }, { status:
201 });
}
```

```
## 4. API CRUD-api/users/[id]/route.js
import { NextResponse } from "next/server";
import connectDB from "@lib/db";
import User from "@models/User";

// GET one
export async function GET(request, { params }) {
  await connectDB();
  const user = await User.findById(params.id);
  if (!user) return NextResponse.json({ success: false, error: "Not
found" }, { status: 404 });
  return NextResponse.json({ success: true, data: user }, { status:
200 });
}
```

```
// PUT update
export async function PUT(request, { params }) {
  await connectDB();
  const body = await request.json();
  const user = await User.findByIdAndUpdate(params.id, body, {
new: true });
  if (!user) return NextResponse.json({ success: false, error: "Not
found" }, { status: 404 });
  return NextResponse.json({ success: true, data: user }, { status:
200 });
}
```

```
// DELETE
export async function DELETE(request, { params }) {
  await connectDB();
  await User.findByIdAndDelete(params.id);
  return NextResponse.json({ success: true, message: "Deleted" }, {
status: 200 });
}
```

```
## 5. Auth - api/auth/register/route.js
import { NextResponse } from "next/server";
import connectDB from "@lib/db";
import User from "@models/User";
import bcrypt from "bcryptjs";
```

```
export async function POST(request) {
  await connectDB();
  const { name, email, password } = await request.json();
  const existing = await User.findOne({ email });
  if (existing) return NextResponse.json({ success: false, error:
"Email exists" }, { status: 400 });
  const hashedPassword = await bcrypt.hash(password, 10);
  await User.create({ name, email, password: hashedPassword });
  return NextResponse.json({ success: true, message: "Registered"
}, { status: 201 });
}
```

```
## 6. Auth — app/api/auth/login/route.js
import { NextResponse } from "next/server";
import connectDB from "@lib/db";
import User from "@models/User";
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
```

```
export async function POST(request) {
  await connectDB();
  const { email, password } = await request.json();
  const user = await User.findOne({ email });
  if (!user) return NextResponse.json({ success: false, error: "Invalid
credentials" }, { status: 401 });
  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) return NextResponse.json({ success: false, error:
"Invalid credentials" }, { status: 401 });
```

```
const token = jwt.sign({ id: user._id, email: user.email },
process.env.JWT_SECRET, { expiresIn: "7d" });
const response = NextResponse.json({ success: true, message:
"Logged in" }, { status: 200 });
response.cookies.set("token", token, { httpOnly: true, maxAge: 60 *
60 * 24 * 7 });
return response;
}
```

```
## 7. Auth - api/auth/logout/route.js
import { NextResponse } from "next/server";
```

```
export async function POST() {
  const response = NextResponse.json({ success: true, message:
"Logged out" });
  response.cookies.set("token", "", { maxAge: 0 }); // clears cookie
  return response;
}
```

```
## 8. Protected Route — app/api/profile/route.js
import { NextResponse } from "next/server";
import jwt from "jsonwebtoken";
import connectDB from "@lib/db";
import User from "@models/User";
```

```
export async function GET(request) {
  const token = request.cookies.get("token")?.value;
  if (!token) return NextResponse.json({ success: false, error: "Not
authenticated" }, { status: 401 });
  const decoded = jwt.verify(token, process.env.JWT_SECRET); // {
id, email }
  await connectDB();
  const user = await User.findById(decoded.id).select("-password");
  // exclude password
  return NextResponse.json({ success: true, data: user }, { status:
200 });
}
```

// File: /middleware.js (Root folder, NOT inside app/)

```
import { NextResponse } from "next/server";
```

```
export function middleware(request) {
  // 1. Extract token from request cookies
  const token = request.cookies.get("token")?.value;

  // 2. Redirect to login if token doesn't exist
  // Edge runtime safe: redirect if token is entirely missing
  if (!token)
    { return NextResponse.redirect(new URL("/login", request.url)); }
  return NextResponse.next();
}
```

```
// 3. Configure which routes trigger this middleware
export const config = {
  matcher: ["/dashboard/:path*", "/profile/:path*"],
};
```

```
## 9. Frontend — app/users/(citizen)/page.js
Route: /users           () is ignored in route
"use client"
import { useState, useEffect } from "react";
import { useRouter } from "next/navigation";

export default function UsersPage() {
  const [users, setUsers] = useState([]);
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const router = useRouter();
```

```
useEffect(() => { fetchUsers(); }, []);
```

```
async function fetchUsers() {
  const res = await fetch("/api/users");
  const data = await res.json();
  setUsers(data.data);
}
```

```
async function handleCreate() {
  await fetch("/api/users", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name, email }),
  });
  fetchUsers();
}
```

```
async function handleDelete(id) {
  await fetch(`/api/users/${id}`, { method: "DELETE" });
  fetchUsers();
}
```

```
return (
  <div>
    <input placeholder="Name" onChange={(e) =>
setName(e.target.value)} />
    <input placeholder="Email" onChange={(e) =>
setEmail(e.target.value)} />
    <button onClick={handleCreate}>Add</button>
    {users.map((u) => (
      <div key={u._id}>
        <p>{u.name} — {u.email}</p>
        <button onClick={() => handleDelete(u._id)}>Delete</button>
      </div>
    ))}
  </div>
);
```

```
## app/layout.js
export default function RootLayout({ children }) {
  return (
    <html>
      <body className="">
        <Navbar />
        <main>{children}</main>
        <Footer />
      </body>
    </html>
  )
}
```

```
// File: app/dashboard/layout.js
// This layout ONLY applies to /dashboard routes
```

```
## Catch-All Routes
// File: app/docs/[...slug]/page.js
app/docs/[...slug]/page.js to make parameters optional and make
app/docs/page.js work...else it will crash 404.
// Matches multiple URL segments like /docs/a/b/c
import { useRouter } from "next/navigation"
```

```
export default async function DocsPage({ params }) {
  const { slug } = await params
  return (
    <div>
      <h1>Docs Page</h1>
      {/* slug is an array here */}
      <p>Path: {slug.join("/ ")}</p>

      // Navigation
      <Link href="/about">Go to About</Link>
    </div>
  )
}
```

```
## 10. JWT Quick Reference
import jwt from "jsonwebtoken";
```

```
jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: "7d"
}); // create
jwt.verify(token, process.env.JWT_SECRET); // verify+decode
response.cookies.set("token", token, { httpOnly: true, maxAge:
60*60*24*7 }); // set cookie
response.cookies.set("token", "", { maxAge: 0 }); // clear cookie
request.cookies.get("token")?.value; // read cookie
```

```
// EXPRESS
res.cookie("token", value, { httpOnly: true });
res.clearCookie("token");
req.cookies.token;
```

// Add this for Server Actions cookie management:

```
import { cookies } from "next/headers";
cookies().set("token", token, { httpOnly: true, maxAge: 60*60*24*7 });
// Set
cookies().get("token")?.value; // Read
cookies().delete("token"); // Clear / Logout
```

// JWT Composition: Header (Alg) + Payload (Data) + Signature (Secret Check)

// Payloads are Encoded, NOT encrypted! Never store sensitive user passwords inside them.

```
## 12. Props Quick Reference
// params — dynamic route: app/users/[id]/page.js
export default function Page({ searchParams }) {
  return <p>{params.id}</p>; // /users/42 -> "42"
}
```

```
// searchParams — query string: /users?sort=asc
export default function Page({ searchParams }) {
  return <p>{searchParams.sort}</p>; // "asc"
}
```

// Regular props — parent to child

```
function Card({ name, age }) { return <p>{name} {age}</p>; }
<Card name="Ali" age={22} />
```

```
## 13. .env.local
MONGO_URI=mongodb://localhost:27017/myapp
JWT_SECRET=anyrandomlongstring123
```

```
## 14. Key Rules
- API files MUST be named route.js
- "use client" must be FIRST line
- useState/useEffect = needs "use client"
- Always await connectDB() before queries
- bcrypt.hash(password, 10) before saving
- bcrypt.compare(plain, hashed) on login
- { new: true } in findByIdAndUpdate = returns updated doc
- .select("-password") = exclude field from result
- useRouter from "next/navigation" not "next/router"
```

// Proper Client-Side Redirects (inside "use client" pages)

```
import { useRouter } from "next/navigation";
```

```
const router = useRouter();
router.push("/dashboard"); // Moves to page instantly without hard
reloading
```

```
## 15. Server Actions Form Extraction
// Place "use server" at the very top of your action file!
export async function handleForm(formData) {
  const email = formData.get("email");
  const password = formData.get("password");
```

// To redirect inside a Server Action or Protected Route:

```
import { redirect } from "next/navigation";
redirect("/dashboard");
}
```

jQuery

```
// Setup
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
$(document).ready(function() {}); // or $(function() {});
```

```
// Selectors
$( "p" ) // tag
$( ".box" ) // class
$( "#main" ) // id
```

```
// DOM
$("#id").text("hello") // get/set text
$("#id").html("<b>hi</b>") // get/set HTML
$("#id").val() // form input value
```

```
$("#id").hide() / .show() / .toggle()
$("#id").addClass("x") / .removeClass("x")
$("#id").css("color", "red")
$("#id").attr("href", "url")
$("<div>new</div>").appendTo("#list")
$("#id").remove()
```

// Events

```
$("#btn").click(function() {});
$("#input").on("keyup", function() {});
$(document).on("click", ".dynamic", fn) // delegated
```

// AJAX

```
$.ajax({ url: "/api", method: "GET",
  success: function(data) {},
  error: function(err) {}
});
$.get("/api", function(data) {});
$.post("/api", { name: "Ali" }, function(data) {});
```

— EXPRESS CORE & DATABASE CRUD —

```
const express = require("express");
const app = express();
app.use(express.json()); // Parse JSON body middleware
```

// --- 1. Request & Response Quick Reference ---

// req.properties: .body (POST data), .params.id (path variable), .query.sort (?sort=asc)

// req.get(headerName) -> Fetches specific incoming header values (e.g., Authorization)

// res.methods: .send() (HTML/text), .json(obj) (sets content-type), .status(code)

// res.type('text') (sets mime-type), res.redirect(url), res.sendStatus(200) (status + default text)

// --- 2. Asynchronous MongoDB MERN Routes ---

```
app.get("/api/users", async (req, res) => {
  try { res.json(await User.find()); } // Fetch all [cite: 1134, 1135]
  catch (e) { res.status(500).json({ error: e.message }); }
});
```

```
app.post("/api/users", async (req, res) => {
  try {
    const user = new User(req.body); // Instantiate document
    await user.save(); // Save to MongoDB instance
    res.status(201).json(user); // Created
  } catch (e) { res.status(500).json({ error: e.message }); }
});
```

```
app.put("/api/users/:id", async (req, res) => {
  try {
    // { new: true } is vital: returns the freshly modified document, not
the old one!
    const user = await User.findByIdAndUpdate(req.params.id,
req.body, { new: true });
    res.json(user);
  } catch (e) { res.status(500).json({ error: e.message }); }
});
```

```
app.delete("/api/users/:id", async (req, res) => {
  try {
    await User.findByIdAndDelete(req.params.id); [cite: 1144]
    res.json({ message: "User deleted" }); [cite: 1145]
  } catch (e) { res.status(500).json({ error: e.message }); }
});
```

```
app.listen(3000, () => console.log("Running"));
```

— EXPRESS MIDDLEWARE TYPES —

Middleware — runs before route handler:

// 1. Application-level: Fires on EVERY incoming server request

```
app.use((req, res, next) => { console.log(req.method, req.url); next();
});
```

// 2. Route-specific: Executes exclusively on designated routing structures

```
app.use('/api', (req, res, next) => { next(); });
```

// 3. Error-handling: Catches application failures (Must accept exactly 4 parameters)

```
app.use((err, req, res, next) => { res.status(500).send("Something
broken"); });
```

— MODULAR EXPRESS ROUTER —

// --- Express Router (Modular files) ---

// File: ./routes/users.js

```
const router = express.Router();
router.get('/', (req, res) => { res.send("Get items"); });
module.exports = router;
```

// File: ./app.js

```
const usersRoute = require('./routes/users');
app.use('/users', usersRoute); // Mounts router onto the path prefix
```

// --- Traditional Express JWT Authentication ---

```
const jwt = require("jsonwebtoken"); [cite: 1130]
const token = jwt.sign({ id: user._id }, secret, { expiresIn: '7d' }); //
Generate token
const payload = jwt.verify(token, secret); // Authenticate & extract
payload data
// HTTP Authorization Header standard format convention: Bearer
<token_string>
```

Status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 500 Internal Server Error

— STORAGE APIs: DEFINITIVE SPECIFICATION —

// --- LOCAL STORAGE (Persists forever across browser restarts & tabs)

```
localStorage.setItem("theme", "dark");
localStorage.getItem("theme");
```

// Storing Objects: Must serialize into strings first using JSON.stringify

```
localStorage.setItem("user", JSON.stringify({ name: "Ali", age: 20 }));
const user = JSON.parse(localStorage.getItem("user")); // Parse
back to object
console.log(user.name);
```

```
localStorage.removeItem("theme"); // Clears single key
localStorage.clear(); // Wipes all local storage keys for the domain
```

// --- SESSION STORAGE (Dies instantly when the individual tab is closed)

```
sessionStorage.setItem("tab_state", "loading");
const state = sessionStorage.getItem("tab_state");
```

```
sessionStorage.removeItem("tab_state"); // Clear single key
sessionStorage.clear(); // Wipe all session storage keys
for the tab
```

Conceptual Comparison Matrix		
Feature	Local Storage	Session Storage
	Stays even after closing the browser and reopening it. Persistent until manually deleted.	Data dies the exact moment you close that specific browser tab.
Data Lifespan		
Tab Isolation	Shared globally across all open tabs of the exact same website.	Isolated. Each tab has its own completely separate session instance.
Page Refresh	Survives page refreshes.	Survives page refreshes within the same tab.

JSON methods

```
JSON.stringify({ name: "Ali" }) // object → string
JSON.parse('{"name":"Ali"}') // string → object
```

```
users.map(u => <p key={u._id}>{u.name}</p>)
users.find(u => u._id === id) // returns first match
users.filter(u => u.age > 18) // returns array
```

-----MONGODB RAW QUERIES -----

document format

```
{ "first name": "Daniyal",
  "address": { "city": "C", "house": "H"}}
```

```
db.users.insertOne({ name: 'Ali', age: 22 })
db.users.insertMany([ {name: 'A', age: 20},
{ name: 'B', age: 25} ])
```

```
db.users.find()
db.users.find({ age: 22 })
db.users.find({ age: { $gt: 20 } })
db.users.find({ city: 'Islamabad', age: 22 })
db.users.find(
  { $or: [{age: 20}, {city: 'Lahore'} ],
  { name: 1, age: 1, _id: 0 }
)
db.users.find().sort({ age: 1 }) // asc
db.users.find().sort({ age: -1 }) // desc
db.users.find().limit(5)
```

```
User.findById(id) // get one by id
User.findOne({ email: 'x@x.com' }) // get one by field
User.create({ name, email }) // insert new
User.findByIdAndUpdate(id, { name }, { new: true }) // update, return new doc
User.findByIdAndDelete(id) // delete by id
```

```
db.users.updateOne({name:'Ali'},{$set:
{age:23}})
db.users.updateMany({age:20},{$set:
{active:true}})
```

```
$set: set field value
$unset: remove field
$inc: increment number
$push: add to array
$pull: remove from array
$currentDate:
```

```
db.users.deleteOne({ name: 'Ali' })
db.users.deleteMany({ age: { $lt: 18 },
{ "trial": { $exists: true, $eq: true } }
{ role: { $nin: ["admin", "moderator"] } },
})
```

// Operators: \$gt \$lt \$gte \$lte \$eq/ne \$in/nin \$and/or/nor/not \$exists

Aggregation / JOIN (\$lookup)

```
db.articles.aggregate([
{$lookup:{from:'User',
localField:'Author',
foreignField:'_id',as:'user'}},
{$unwind:'$user'},
{$project:{title:1,
Author:'$user.Name'}}
])
```

Indexing

```
db.users.createIndex({lastName:1})
db.users.createIndex({email:1,
{unique:true, name:'EmailIdx'})
```

— MONGODB CONNECTION + MODEL —

```
const mongoose = require('mongoose');
mongoose.connect('mongodb://127.0.0.1:27017/
myDB')
.then(() => console.log('Connected'))
.catch(err => console.log(err));
const userSchema = new mongoose.Schema({
  name: String, age: Number, email: String
});
const User = mongoose.model('User',
userSchema);
```

— REACT FULL APP (CLIENT SIDE) —

```
const App = () => {
  const [users, setUsers] = useState([]);
  const [name, setName] = useState("");
  const [age, setAge] = useState("");
```

```
useEffect(() => { fetchUsers(); }, []);
```

```
const fetchUsers = async () => {
  const res = await fetch('/api/users');
  const data = await res.json();
  setUsers(data);
};
```

```
const addUser = async () => {
  const res = await fetch('/api/users', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ name, age: Number(age) })
  });
  const newUser = await res.json();
  setUsers([...users, newUser]);
};
```

```
const updateUser = async (id) => {
  await fetch(`/api/users/${id}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ name, age: Number(age) })
  });
  fetchUsers();
};
```

```
const deleteUser = async (id) => {
  await fetch(`/api/users/${id}`, { method: 'DELETE' });
  setUsers(users.filter(u => u._id !== id));
};
```

```
return (
  <div>
    <input value={name} onChange={e => setName(e.target.value)}
placeholder='Name' />
    <input value={age} onChange={e => setAge(e.target.value)}
placeholder='Age' />
    <button onClick={addUser}>Add</button>
```

```
    {users.map(user => (
      <div key={user._id}>
        {user.name} - {user.age}
        <button onClick={() =>
updateUser(user._id)}>Update</button>
        <button onClick={() => deleteUser(user._id)}>Delete</button>
      </div>
    ))}
  </div>
);
```

— REACT EXTRAS —

```
// --- Conditional Rendering ---
{loading && <p>Loading...</p>}
{isLoggedIn ? <Dashboard /> : <Login />}
```

// --- Multi-Level Props & Parent-to-Child Architecture ---

```
// 1. ROOT PARENT (App.js): Declares state data and state modifier
callbacks
const App = () => {
  const products = [{ id: 1, name: 'Laptop', price: 800 }, { id: 2, name:
'Phone', price: 500 }];
  const [order, setOrder] = useState([]);
```

```
  const addToOrder = (product) => setOrder((prevOrder) =>
[...prevOrder, product]);
```

```
  return (
    <div>
      {/* Passing data and state-modifier functions down as props */}
      <ProductList products={products} addToOrder={addToOrder} />
      <OrderSummary order={order} />
    </div>
  );
```

```
// 2. INTERMEDIATE CHILD (ProductList.js): Receives props,
iterates data, passes props down again
const ProductList = ({ products, addToOrder }) => (
  <div>
    {products.map((product) => (
      // Forwarding props down another level. Key prop MUST sit at
the top of the map block loop!
      <ProductItem key={product.id} product={product}
addToOrder={addToOrder} />
    ))}
  </div>
);
```

```
// 3. LEAF CHILD (ProductItem.js): Receives props and binds the
parent callback trigger to DOM events
const ProductItem = ({ product, addToOrder }) => (
  <div>
    <h3>{product.name} - ${product.price}</h3>
    <button onClick={() => addToOrder(product)}>Add to
Order</button>
  </div>
);
```

// --- Lists & Keys (Keys must be unique siblings) ---

```
const items = ['Apple', 'Banana', 'Cherry'];
<ul>
  {items.map((item, i) => <li key={i}>{item}</li>)}
</ul>
```

```
// --- Advanced List Aggregation & Conditional Rendering ---
const OrderSummary = ({ order }) => {
  // .reduce computes a single numerical value from array data.
Syntax: (accumulator, element) => implementation, startingValue
const total = order.reduce((sum, item) => sum + item.price, 0);
```

```
  return (
    <div>
      {/* Inline short-circuit check to render an empty collection
placeholder message */}
      {order.length === 0 ? (
        <p>No items in order</p>
      ) : (
        <ul>
          {/* Explicit index map backup strategy if documents lack
distinct unique ID primary keys */}
          {order.map((item, index) => (
            <li key={index}>{item.name} - ${item.price}</li>
          ))}
```

```
        </ul>
      )}
    </div>
    <h3>Total Calculated Accumulation: ${total}</h3>
  </div>
);
```

```
// --- Controlled Forms (State drives input value) ---
const [name, setName] = useState("");
<input value={name} onChange={e => setName(e.target.value)} />
<button onClick={handleSubmit}>Send</button>
```

```
// --- Uncontrolled Forms (DOM retains its own input state via Ref) ---
const inputRef = React.createRef();
<input type="text" ref={inputRef} /> // Access via:
inputRef.current.value
```

```
// --- Vanilla Promise Fetch Chain ---
fetch('http://api.example.com/users')
.then(res => res.json())
.then(data => setState({ data }))
.catch(console.log);
```

— REACT HOOKS —

```
const [count, setCount] = useState(0);
```

```
// --- useEffect Dependency Array Configurations ---
useEffect(() => { fetchUsers(); }, []); // Run style 1: On mount only
useEffect(() => { fetch(); }, [id]); // Run style 2: On mount + when
'id' state changes
useEffect(() => { fetch(); }); // Run style 3: Triggers on EVERY
single render
```

```
// --- useLayoutEffect (Executes synchronously BEFORE browser
paints elements) ---
// Primarily utilized for immediate physical DOM measurements
useLayoutEffect(() => {
  const h = divRef.current.offsetHeight;
  }, []);
```

```
// --- useRef (Persistent mutable reference object; does not trigger
re-renders) ---
const inputRef = useRef(null);
inputRef.current.focus();
<input ref={inputRef} />
```

```
// --- useContext (Global Data Sharing Layer without Prop Drilling) ---
const ThemeContext = createContext();
// Parent Provider:
<ThemeContext.Provider value='dark'>
  <Child />
</ThemeContext.Provider>
// Child Consumer:
const theme = useContext(ThemeContext);
```

```
// --- useReducer (Alternative hook for managing complex local
component state) ---
const reducer = (state, action) => {
  switch(action.type) {
    case 'inc': return { count: state.count + 1 };
    case 'dec': return { count: state.count - 1 };
    default: return state;
  }
};
const [state, dispatch] = useReducer(reducer, { count: 0 });
dispatch({ type: 'inc' });
```

```
// --- useMemo (Memoizes/Caches a computed mathematical
VALUE) ---
const total = useMemo(() => {
  return nums.reduce((a, n) => a + n, 0);
}, [nums]);
```

```
// --- useCallback (Memoizes/Caches an entire inline FUNCTION
instance) ---
const handleClick = useCallback(() => {
  doSomething(id);
}, [id]);
```

```
// --- Core Hook Differences (High Exam Probability) ---
// useMemo -> Caches a calculated VALUE -> useMemo(() =>
calc(), [d])
// useCallbck -> Caches a raw FUNCTION setup -> useCallback(()
=> fn(), [d])
```

```
// useEffect -> Asynchronous -> Runs AFTER painting elements ->
Best for general API fetches
// useLayoutEffect -> Synchronous -> Runs BEFORE painting
elements -> Best for manual DOM changes
```

— CLASS COMPONENTS & LIFECYCLES —

```
class Counter extends React.Component {
```

```
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }
  render() {
    return (
      <button onClick={() => this.setState({ count: this.state.count + 1
})}>
```

```
        {this.state.count}
      </button>
    );
  }
}
```

```
// --- Class Lifecycle Mapping to Hooks ---
// componentDidMount: Fires directly after first render (Ideal for API
resource calls)
// componentDidUpdate: Fires immediately following any state
adjustment re-render
// componentWillUnmount: Cleanup loop executed before
component removal (Clear intervals/listeners)
componentDidMount() {
  fetch('/api/data')
  .then(r => r.json())
  .then(d => this.setState({ data: d }));
}
```

— CLIENT-SIDE NAVIGATION & ROUTING (React-Router-Dom) —

```
// --- Declarative View Tree Configuration ---
<Router>
  <Link to="/about">About</Link>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
    <Route path="*" element={<NotFound />} />
  </Routes>
</Router>
```

```
// --- Programmatic Navigation Redirection API ---
const navigate = useNavigate();
navigate('/dashboard');
```

```
// --- Parameter Dynamic Sub-Route Parsing ---
// Mapping route definition layout:
<Route path="/user/:id" element={<User />} />
// Extracted inside target route component file:
import { useParams } from 'react-router-dom';
const { id } = useParams();
```

— CLIENT AXIOS LAYER —

```
// --- Axios GET Request Execution Flow ---
sessionStorage.setItem("session_tab", "active_id_99"); // Erased
automatically when tab closes
const tabId = sessionStorage.getItem("session_tab");
```

```
// --- Axios POST Request Execution Flow ---
axios.post('/api', { name, msg })
.then(() => navigate('/'));
```

— CONTEXT API REFERENCE —

```
const ThemeCtx = React.createContext('light');
```

```
// Provider Framework (Parent Root Wrapper Component Context):
<ThemeCtx.Provider value='dark'>
  <App />
</ThemeCtx.Provider>
```

```
// Consumer Hook Framework (Nested Child Evaluation Context
Target):
const theme = useContext(ThemeCtx);
```

Next.js Props Concept vs React Props

```
// React (Client Routing via hooks):
const { id } = useParams();
```

```
// Next.js App Router (Server-Side Injection Prop Promises):
export default async function Page({ params, searchParams }) {
  const { id } = await params; // From folder paths like
app/user/[id]/page.js
  const { sort } = await searchParams; // From query query strings
like /user/12?sort=asc
  return <p>{id} - {sort}</p>;
}
```

Next.js Server Components vs. Client Components

Feature	Server Components (Default)	Client Components ("use client")
---------	-----------------------------	----------------------------------

Execution	Runs only on the server. Zero client JS impact.	Prerendered on server, hydrates in browser.
Data Fetching	Can be directly marked async to await DB models.	Must fetch endpoints asynchronously via useEffect/Axios.
Security	Safe to read sensitive env keys & write direct database calls.	Exposes bundle logic to the web. Must use secure APIs.
Hooks	Cannot use useState, useEffect, or useRouter.	Can freely use state management, effects, and browser context hooks.

// --- Server Component (Direct DB Query Pattern) ---
import connectDB from "@lib/db";
import Property from "@models/Property"; // Real Estate CRM example hint

```
export default async function RealEstateDashboard() {
  await connectDB();
  const listings = await Property.find({ status: "active" }); // Direct
secure lookup
  return (
    <main>
      {listings.map(p => <div key={p._id}>{p.title} - ${p.price}</div>)}
    </main>
  );
}
// Interactivity Layer: Client Component (Must add "use client" as line
1)
// Required whenever using: useState, useEffect, onClick handlers,
or useRouter hooks.
```

Git & Vercel Deployment Terminal Commands

```
# --- Git Lifecycle Management ---
git init # Spin up local version tracking repository
git add . | git commit -m "hehe" | git push origin main
```

```
# --- Vercel Production Deployments ---
npm run build # Compiles frontend locally to verify zero build-
breaking syntax bugs
vercel # Uploads project instance files to an ephemeral
preview test URL
vercel --prod # Promotes changes directly to the live
production server domain
```

Missing Browser APIs & Array Methods Reference

```
// --- Session Storage Utilities ---
sessionStorage.setItem("session_tab", "active_id_99"); // Erased
automatically when tab closes
const tabId = sessionStorage.getItem("session_tab");
```

```
// --- Advanced Array Lookups (MERN Stack Basics Hint) ---
const products = [{ id: 1, type: "Laptop" }, { id: 2, type: "Phone" }];
// .find() returns the FIRST matching object element instance itself:
const foundItem = products.find(p => p.id === 2); // Result: { id: 2,
type: "Phone" }
// .filter() always returns a newly constructed array block containing
all matches:
const filteredItems = products.filter(p => p.id === 2); // Result: [{ id:
2, type: "Phone" }]
```

Axios Config Reference (MERN Basics Hint)

```
// Axios Base Configuration with Global Security Headers Injection
import axios from "axios";
```

```
const crmApi = axios.create({
  baseURL: "http://localhost:3000/api",
  timeout: 5000,
  headers: { "Content-Type": "application/json" }
});
```

```
// Programmatic Request Token Authentication Interception
crmApi.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`; // Matches
'Bearer <token>' standard format
  }
  return config;
});
```

Promise Execution Patterns & Outputs

```
// 1. Sequential Execution (Slower - wait for one to finish before
starting next)
async function sequential() {
  const a = await task1(); // Takes 2 seconds
```

```
const b = await task2(); // Takes 3 seconds -> Total time: 5
seconds
return [a, b];
}
```

```
// 2. Parallel Execution (Faster - runs both at the exact same time)
async function parallel() {
  // Promise.all fails instantly if ANY promise rejects!
  const [a, b] = await Promise.all([task1(), task2()]); // Total time: 3
seconds
  return [a, b];
}
```

```
// 3. Common Gotcha: Forgotten Await
const data = fetch("/api/users"); // WRONG! Returns a pending
Promise object, not data.
const data = await fetch("/api/users"); // CORRECT! Resolves into
the Response stream.
```

Promises / .then() chain

```
fetch("/api/users")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));

// Promise.all — runs multiple at once
Promise.all([fetch("/api/a"), fetch("/api/b")])
  .then(([r1, r2]) => Promise.all([r1.json(), r2.json()]))
  .then(([d1, d2]) => console.log(d1, d2));
```

— AXIOS VS FETCH: DEFINITIVE SPECIFICATION —

Asynchronous Await Patterns (async/await)
// --- FETCH (Native Browser API) ---
const res = await fetch("/api/users");
const data = await res.json(); // Must manually parse the stream
response object

```
const res = await fetch("/api/users", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Ali" }) // Requires explicit
stringification
});
const data = await res.json();
```

// --- AXIOS (Third-Party Library) ---
const res = await axios.get("/api/users");
console.log(res.data); // Data payload is automatically mapped to
res.data

```
const res = await axios.post("/api/users", { name: "Ali" }); // Auto-
stringifies body payloads
console.log(res.data);
```

```
// Custom Instance Generation
const api = axios.create({ baseURL: "https://api.example.com" });
const res = await api.get("/users");
```

Declarative Promise Chains (.then() / .catch())
// --- FETCH (Native Browser API) ---
fetch('/api/users')
 .then(res => {
 // 401/404 errors DO NOT trigger .catch automatically in Fetch!
 Must check res.ok manually.
 if (!res.ok) throw new Error("401/404 does not trigger .catch
automatically!");
 return res.json();
 })
 .then(data => console.log(data));

// --- AXIOS (Third-Party Library) ---
axios.get('/api/users')
 .then(res => {
 // 1. Automatically throws runtime errors for any non-2xx status
codes (triggers .catch)
 // 2. Data payload is already parsed into native JavaScript objects
under 'res.data'
 console.log(res.data);
 }).catch(err => console.log(err.response.status));